Übungsblatt 7: Timing messen und Beobachtungen auswerten

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Was kostet die Beobachtung? (8 Punkte)

Eine Funktion benötigt im Lehrmodell 18 µs eigene Arbeit. Anschließend sendet sie die sechs ASCII-Zeichen `READY\n` bei 230400 bit/s, 8N1, ohne Nullterminator. Bei diesem blockierenden Lehrfall liegt die volle Leitungsdauer zusätzlich innerhalb der Antwort; andere Kosten sind null. Abschlussfrist: 240 µs.

1. Berechnen Sie Bitzahl und reine Leitungsdauer. (2 Punkte)
2. Berechnen Sie Antwortzeit und Fristabweichung. (2 Punkte)
3. Weshalb ist bei DMA nicht dieselbe Leitungsdauer automatisch eigene CPU-Zeit? Welche Restkosten bleiben? (2 Punkte)
4. Definieren Sie eine Messung der ursprünglichen Funktion ohne Ausgabe im Fenster. Warum kann selbst ein Zählerfenster fremde Arbeit enthalten? (2 Punkte)

Musterlösung: Sechs Bytes zu zehn Bit sind 60 Bit, Dauer $60/230400\,\mathrm{s} \approx 260{,}417\,\mu\mathrm{s}$. Antwort 278,417 µs, Verletzung um 38,417 µs. DMA kann senden, während der Kern anderweitig arbeitet; Einrichtung, Formatierung, Pufferbesitz und Abschlussbehandlung bleiben. Anfang/Ende und Quelle definieren, Ausgabe nach der Endmarke; ihre Wirkung auf Folgejobs trotzdem benennen. Ein fortlaufender Kernzähler zählt dazwischenliegende Interrupts und andere Taskarbeit mit, nicht nur eigene Funktionsarbeit.

Aufgabe 2: Jobantwort und Interruptkette (8 Punkte)

1. Ein Job wird bei 4 ms freigegeben, läuft 6–9 und 12–14 ms und ist bei 14 ms fertig; Deadline bei 16 ms. Bestimmen Sie eigene CPU-Zeit, Startverzug, Antwortzeit und Reserve. (4 Punkte)
2. Gemeinsame Modelluhr in µs: Ereignis 200, ISR-Eintritt 207, Task Ready 214, ISR-Ende 221, Taskstart 229, Reaktion 242. Berechnen Sie Eintrittslatenz, ISR-Intervall, Ready-Warten und Gesamtreaktion. Geben Sie eine Zerlegung ohne Überlappung an. (4 Punkte)

Musterlösung: CPU-Zeit 5 ms, Startverzug 2 ms, Antwortzeit 10 ms, Reserve 2 ms. Anfangswarten 2 plus Verdrängung 3 plus eigene Arbeit 5 ergeben 10 ms. Interruptkette: 7 µs Eintritt, 14 µs ISR-Intervall, 15 µs Ready-Warten, 42 µs Gesamtreaktion. Disjunkt etwa $7 + 14 + 8 + 13 = 42$ µs; Ready-Warten 214–229 überlappt die ISR 214–221 und wird nicht zusätzlich zur vollen Zerlegung addiert.

Aufgabe 3: Modulo, Takt und Stichprobe (8 Punkte)

1. Ein bestätigter lokaler 32-Bit-Zähler startet bei `0xFFFFFF80` und endet bei `0x00000050`. Berechnen Sie Modulo-Differenz und Zeit bei stabilen 160 MHz. Wie lange dauert ein voller Umlauf, und welche zusätzliche Annahme braucht die Differenz? (3 Punkte)
2. 2400 Takte werden einmal bei 160 und einmal bei 320 MHz gezählt. Rechnen Sie um. Darf ein M4-Endwert ohne Zeitkorrelation von einem M7-Anfangswert abgezogen werden? (2 Punkte)
3. Vollständige Antwortintervalle betragen 19, 22, 20, 27 und 21 µs. Bestimmen Sie Minimum, Maximum, Mittelwert und Spanne. Prüfen Sie 25 µs Frist. Beweist oder widerlegt die Stichprobe eine Garantie? (3 Punkte)

Musterlösung: Modulo-Differenz $128 + 80 = 208$ Takte, Zeit 1,3 µs. Umlauf $2^{32}/(160 \cdot 10^6) = 26{,}8435456$ s. Der tatsächlich verstrichene Bereich muss kürzer als ein Umlauf sein, bei stabilem Takt und laufendem Zähler. 2400 Takte ergeben 15 bzw. 7,5 µs. Verschiedene lokale Zähler benötigen Skalierung und korrelierte Nullpunkte. Stichprobe: Minimum 19, Maximum 27, Mittelwert $109/5 = 21{,}8$ µs, Spanne 8 µs. Ein gültiger 27-µs-Fall verletzt die Antwortfrist um 2 µs. Er widerlegt die Garantie unter den Bedingungen; günstige Stichproben beweisen umgekehrt keine allgemeine WCET oder Antwortgrenze.

Aufgabe 4: Tracequelle und Datenqualität (8 Punkte)

1. Ein separater Shared-Datensatz hat vier 32-Bit-Felder bei Offsets 0, 4, 8 und 12. Die Basis ist `0x38008000`, der gesuchte Wert das letzte Feld. Welche Adresse muss ein Schreibfilter beobachten? Beweist ein M7-Store bei Write-Back-Cache automatisch die M4-Sicht? (2 Punkte)
2. Ein IRQ läuft, `pxCurrentTCB` verweist weiter auf Task X. Ist das X-CPU-Zeit? Was unterscheidet Taskliste, Running-Profil und Zustandshistorie? (2 Punkte)
3. 120000 Traceereignisse/s zu acht Byte, Abfluss 720000 Byte/s, anfangs 48 KiB frei; konstante Raten ohne Overhead. Bestimmen Sie Füllzeit und Wirkung doppelter Kapazität. (2 Punkte)
4. Eine Verlustmarke trennt zwei Abschnitte; danach ist neue Synchronisation mit Kontext bestätigt. Darf man die Lücke interpolieren oder fehlenden Anfang null setzen? Wie wird der spätere vollständige Abschnitt behandelt? (2 Punkte)

Musterlösung: Filteradresse `0x3800800C`; Quelle, tatsächliches Image, Breite und Richtung prüfen. Ein lokales Storeereignis beweist keine Kohärenz oder M4-Lesung. Während des IRQ läuft Exceptioncode, nicht notwendig X; die Taskliste ist eine Momentaufnahme, Running-Profil braucht CPU-/Exceptionzuordnung, Zustandshistorie zusätzliche Ready-/Blocked-Ereignisse. Eingang 960000 Byte/s, Rückstau 240000 Byte/s, Kapazität 49152 Byte, Füllzeit 0,2048 s; doppelte Kapazität 0,4096 s, keine dauerhafte Lösung. Fehlendes bleibt unbekannt, keine erfundenen Nullwerte/Interpolation. Später bestätigte vollständige Intervalle separat auswerten, keine Jobpaare über die Lücke konstruieren.

Aufgabe 5: Ein Leerfenster ist kein universeller Abzug

Leerfenster enthalten 30, 36 und 33 Takte, Arbeitsfenster 480, 560 und 520 Takte. Darf von jedem Arbeitsfenster 30 abgezogen und der größte Rest „WCET“ genannt werden? Nennen Sie zwei unabhängige Gründe.

Musterlösung: Nein. Das kleinste fremde Leerfenster ist nicht nachgewiesen der jeweilige Messaufwand jedes Arbeitsfensters. Außerdem können Unterbrechungen enthalten sein und die endliche Auswahl deckt nicht alle zulässigen Ausführungen ab. Der größte Rest 530 Takte ist deshalb keine sichere WCET der Funktion.

Aufgabe 6: Die eigene Zählerauswertung implementieren

Programmieren, etwa 30–40 Minuten. Die vorbereitete CM7-Basis führt fünf EMPTY- und fünf WORK-Fenster aus; danach schreibt niemand mehr in `lab07_records`. Kein Traceadapter erforderlich. Der DWT-Zähler und der tatsächliche Kerntakt werden vorab geprüft. Sie implementieren `Lab07_Summarize()` in `summary_student.c`.

1. Prüfen Sie Pointer, Anzahl höchstens `LAB07_CAPACITY` und Auswahl EMPTY/WORK. Berücksichtigen Sie nur die gewählte Art. Ohne passenden Datensatz ist das Ergebnis ungültig, nicht ein künstlicher Nullzeitwert.
2. Bestimmen Sie Anzahl, Minimum, Maximum und 64-Bit-Summe der `cycles_mod32`. Übernehmen Sie `core_hz` aus dem ersten passenden Datensatz und weisen Sie null oder unterschiedliche Takte innerhalb der Gruppe zurück. Die Basis sichert die Messfensteranahmen.
3. Geben Sie Erfolg eins oder Fehler null zurück; auf Fehler bleibt die Ausgabe unverändert. Die Basis legt das Ergebnis in `lab07_summary` ab; **LD1/PB0** leuchtet nur bei erfolgreicher Analyse.
4. Rechnen Sie den Mittelwert erst nach dem Versuch mit $\bar{t} = \text{sum}/(\text{count} \cdot f)$ um. Logging gehört nicht ins Messfenster. Vergleichen Sie beide Arten getrennt und nennen Sie die Grenzen der Aussage.
5. Testen Sie zusätzlich auf Papier oder im Debugger: leere Gruppe, abweichender Takt und eine Summe oberhalb von 32 Bit.

Vorbereitete Basis: Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Board und USB/ST-LINK genügen. Eigene Implementierung und tatsächlichen Prüfstatus festhalten; Debuggerdiagnosen bei angehaltenem zuständigem Kern lesen.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS und gegebenenfalls CMSIS-RTOS2. Papierwerte sind Lehrannahmen.

Musterlösung:

Musterlösung zu Aufgabe 6

Eine private Teilsumme wird erst nach vollständiger Prüfung veröffentlicht. `uint64_t` verhindert einen Summenüberlauf für höchstens 16 32-Bit-Differenzen. Die Auswertung heilt keine mehrfachen Zählerumläufe oder Zählerhalte im ursprünglichen Fenster.

```
#include "lab07_summary.h"

int Lab07_Summarize(const volatile Lab07_Record *records, uint32_t count,
                    Lab07_Kind kind, Lab07_Summary *out)
{
    if (records == NULL || out == NULL || count > LAB07_CAPACITY ||
        (kind != LAB07_EMPTY && kind != LAB07_WORK)) { return 0; }
    Lab07_Summary s = {0};
    for (uint32_t i = 0U; i < count; ++i) {
        if (records[i].kind != (uint32_t)kind) { continue; }
        const uint32_t cycles = records[i].cycles_mod32;
        const uint32_t hz = records[i].core_hz;
        if (hz == 0U || (s.count != 0U && hz != s.core_hz)) { return 0; }
        if (s.count == 0U) {
            s.minimum = s.maximum = cycles;
            s.core_hz = hz;
        }
        if (cycles < s.minimum) { s.minimum = cycles; }
        if (cycles > s.maximum) { s.maximum = cycles; }
        s.sum += cycles;
        ++s.count;
    }
    if (s.count == 0U) { return 0; }
    *out = s;
    return 1;
}
```

Der Mittelwert wird in Gleitkomma oder ausreichend breiter Rechnung gebildet; insbesondere das Produkt aus Count und Frequenz nicht unbeabsichtigt in 32 Bit überlaufen lassen. Die LED bestätigt die Analysefunktion dieses Datensatzes, keine harte Frist.